GÜMÜŞHANE ÜNİVERSİTESİ
İKTİSADİ ve İDARİ BİLİMLER FAKÜLTESİ
YÖNETİM BİLİŞİM SİTEMLERİ BÖLÜMÜ

YBS 207 Algoritma ve Programlamaya Giriş

Set ve Dict Data Types

Öğr. Gör. Dr. Baki GÖKGÖZ

Giriş

- **Set**
  - Küme (Set) Tanımı
    - Benzersiz (tekil) elemanlardan oluşan bir koleksiyondur.
    - Tekrarlayan (aynı) değerlere izin vermez.
    - **Sırasızdır:** elemanların indeksleri veya tanımlı bir sıralaması yoktur.
  - Küme (Set) Özelliklerinin Liste (List) / Demet (Tuple) ile Karşılaştırılması

Özellik	List/Tuple	Set
Sıralı	Evet	Hayır
Eleman Tekrarı	Evet	Hayır
Indeks	Evet	Hayır

Giriş

- **Set**
  - Küme (Set) Tanımı
    - Benzersiz (tekil) elemanlardan oluşan bir koleksiyondur.
    - Tekrarlayan (aynı) değerlere izin vermez.
    - **Sırasızdır:** elemanların indeksleri veya tanımlı bir sıralaması yoktur.
- **Küme (Set) Özelliklerinin Liste (List) / Demet (Tuple) ile Karşılaştırılması**
  - Matematiksel Temel
    - Kümeler, matematiksel kümelere karşılık gelir ve klasik küme işlemlerinin kolayca uygulanmasını sağlar:
      - Birleşim (Union): İki kümenin elemanlarını, tekrar olmadan birleştirir.
      - Kesişim (Intersection): Her iki kümede ortak olan elemanlar.
      - Fark (Difference): Bir kümede olup diğerinde olmayan elemanlar.
      - Simetrik Fark (Symmetric Difference): Yalnızca birinde bulunan, her ikisinde birden bulunmayan elemanlar.
  - Bu işlemler, benzersizlik ve üyelik kontrolü gerektiren algoritma tasarımı ve problem çözme süreçlerinde büyük kolaylık sağlar.
  - Elemanların sırasının önemli olmadığı durumlarda kümeler (set) kullanışlıdır.
  - Python'da set'ler değiştirilebilir (mutable) olduğu için elemanlar dinamik olarak eklenip çıkarılabilir.

Giriş

- **Örnekler**
  - **Örnek:**
  - Tekrarlı elemanlar içeren bir listeden küme (set) oluşturma

```
my_list = [1, 2, 2, 3, 4, 4]
my_set = set(my_list)
print(my_set)
# Output: {1, 2, 3, 4}
```

  - Tekrarlı değerler içeren bir listeyi, tekrarları otomatik olarak ortadan kaldıran bir kümeye (set) dönüştürür.
  - **Örnek: Küme İşlemleri Örneği**

```
a={1,2,3,4}
b={3,4,5,6}
print("Birleşim..:",a|b)
#output Birleşim: {1,2,3,4,5,6}
print("Kesişim..:",a&b)
#output Kesişim: {3,4}
print("Fark(a-b)..:",a-b)
#output Fark(a-b)..: {1,2}
print("Simetrik Fark:", a ^ b)
# Simetrik Fark: {1, 2, 5, 6}
```

Giriş

- **Örnekler**
  - **Örnek:**
  - Bir kümeye (set) eleman ekleme ve kümeden eleman çıkarma

```
s = {1, 2, 3}
s.add(4)
print(s)
# Output: {1, 2, 3, 4}

s.remove(2)
print(s)
# Output: {1, 3, 4}
```

Sets

- **Küme (Set) Tanımı:**
  - Set:
    - süslü parantezler `{}` içinde gösterilen, birbirinden farklı elemanlardan oluşan bir koleksiyondur.
  - Örnek: `{1, 2, 3, 4`
- **Kümelerin (Set) Özellikleri**
  - Benzersiz elemanlar: Tekrarlanan değerler otomatik olarak kaldırılır.
  - Sırasızdır: Kümeler ekleme sırasını korumaz; indeksleme veya dilimleme desteklemez. Elemanlar öngörülemeyen bir sırada saklanır ve gösterilir.
  - Heterojen: Kümeler farklı veri türlerinden elemanlar içerebilir (tamsayı, ondalık sayı, string, boolean, tuple).
  - Değiştirilebilir (Mutable): Kümeler oluşturulduktan sonra eleman eklenip çıkarılarak değiştirilebilir

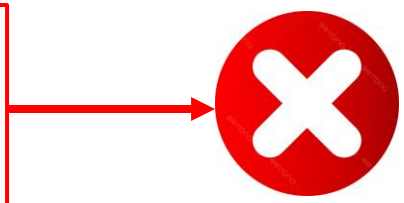
Sets

- **Süslü parantezler {} ve set() fonksiyonu ile küme (set) oluşturma**
  - Örnek:

```
S1 = {1, 2, 3, 4, 5, 6}
S2 = set([1, 2, 3, 4, 5, 6])
S3 = set('python') S4 = {5}
print(type(S1))
# <class 'set'>
print(S3)
# e.g. {'h', 'n', 'o', 'p', 't', 'y'}
```

- **Boş küme oluşturma – doğru ve yanlış kullanım**
  - Örnek:

```
empty_set={}
Print(type(empty_set))
# output <class 'dict'>
```



```
empty_set=set()
Print(type(empty_set))
# output <class 'set'>
```



Sets

- **Bir döngü içinde elemanları yazdırarak set'in sırasız (unordered) yapısını gösterme**
  - Örnek:

```
S = {10, 20, 30, 40, 50}
for elem in S:
        print(elem)
```

  - Çıktı sırası rastgeledir ve ekleme sırasından farklı olabilir.
- **Bir kümeye (set) eleman ekleme**
  - Örnek

```
S = {10, 20, 30, 40, 50}
S.add(100)
# integer ekleme
S.add('merhaba')
# string ekleme
S.add((1, 2, 3))
# tuple ekleme
print(S)
```

Sets

- **Bir kümeden (set) eleman silme ve pop() ile eleman çıkarma**
  - Örnek:

```
S = {10, 20, 30, 40, 50, 60}
S.remove(20)
# Removes 20 print(S)

popped = S.pop()
print("Alınan Eleman..::", popped)
print(S)
```

  - Çıktı sırası rastgeledir ve ekleme sırasından farklı olabilir.

Dictionary

- **Sözlük (Dictionary) Genel Bakış**
  - Dictionary:
    - anahtar–değer `(key-value)` çiftlerinden oluşan bir koleksiyondur.
  - Anahtarlar benzersiz olmalı ve değiştirilemez (immutable) veri tiplerinden seçilmelidir (ör. string, sayı, tuple).
  - Değerler herhangi bir veri tipinde olabilir ve tekrarlanabilir.
  - Dictionary hızlı erişim (lookup) sağlar;
    - bu da onları sık arama gerektiren senaryolar için uygun kılar.
  - **İç Çalışma Yapısı**
    - Bir sözlük (dictionary), anahtar ve değerleri saklamak için dahili olarak bir hash tablosu kullanır.
    - Anahtarın hash değeri, değerin nerede saklanacağını belirler ve bu sayede hızlı erişim (retrieval) sağlanır.

Dictionary

- **Sözlük (Dictionary) Oluşturma**
  - Anahtar–değer çiftleriyle süslü parantezler {} kullanılarak oluşturulur:

```
my_dict = {'key1': 'value1', 'key2': 'value2'}
```

  - `dict()` yapıcı (constructor) fonksiyonunu kullanma:

```
my_dict = dict(key1='value1', key2='value2')
```

  - Sözlük kavramaları (dictionary comprehensions) – sözlük oluşturmanın kısa ve verimli yolu:

```
squared = {x: x*x for x in range(5)}
print(squared)
```

Dictionary

- **Sözlük (Dictionary) Metotları (yaygın kullanılanlar)**
  - **`dict.keys()`:** Tüm anahtarların bir görünümünü (view) döndürür.
  - **`dict.values()`:** Tüm değerlerin bir görünümünü döndürür.
  - **dict.items():** (anahtar, değer) çiftlerinin bir görünümünü döndürür.
  - **dict.get(key, default):** Belirtilen anahtara karşılık gelen değeri döndürür; anahtar bulunamazsa default değerini verir.
  - **dict.update(other_dict):** Başka bir sözlükteki anahtar–değer çiftleriyle sözlüğü günceller.
  - **dict.pop(key):** Belirtilen anahtarı siler ve karşılık gelen değerini döndürür.
  - **dict.clear():** Sözlükteki tüm elemanları siler.

Dictionary

- **Örnekler**
  - **Örnek:**
  - Tamsayı anahtarlar ve string değerler kullanarak sözlük (dictionary) oluşturma

```
D1 = {1: "bir", 2: "iki", 3: "üç", 4: "dört"}
print(D1)
# Output: {1: 'bir', 2: 'iki', 3: 'üç', 4: 'dört'}
```

  - **Örnek:**
  - Anahtar ile Sözlük Değerine Erişim

```
print(D1[1])
# Output: bir
```

  - **Örnek:**
  - Var olan anahtarın değerinin değiştirilmesi

```
D1[3] = "güncel üç"
print(D1)
# Output: {1: 'bir', 2: 'iki', 3: 'güncel üç', 4: 'dört'}
```

Dictionary

- **Örnekler**
  - **Örnek:**
  - Yeni bir **key–value** çifti ekleme

```
D1 = {1: "bir", 2: "iki", 3: "üç", 4: "dört"}
D1[5] = "beş"
print(D1)
# Output:{1: 'bir', 2: 'iki', 3: 'üç', 4: 'dört', 5: 'beş'}
```

  - **Örnek:**
  - Sözlük Üzerinde Gezinme: Anahtar ve Değerleri Yazdırma

```
for key in D1:
print(f"Key: {key}, Value: {D1[key]}")
# Output:
# Key: 1, Value: bir
# Key: 2, Value: iki
# Key: 3, Value: üç
# Key: 4, Value: dört
```